Differential Privacy

In Federated Learning

Bryan Cardenas
High Performance Machine Learning

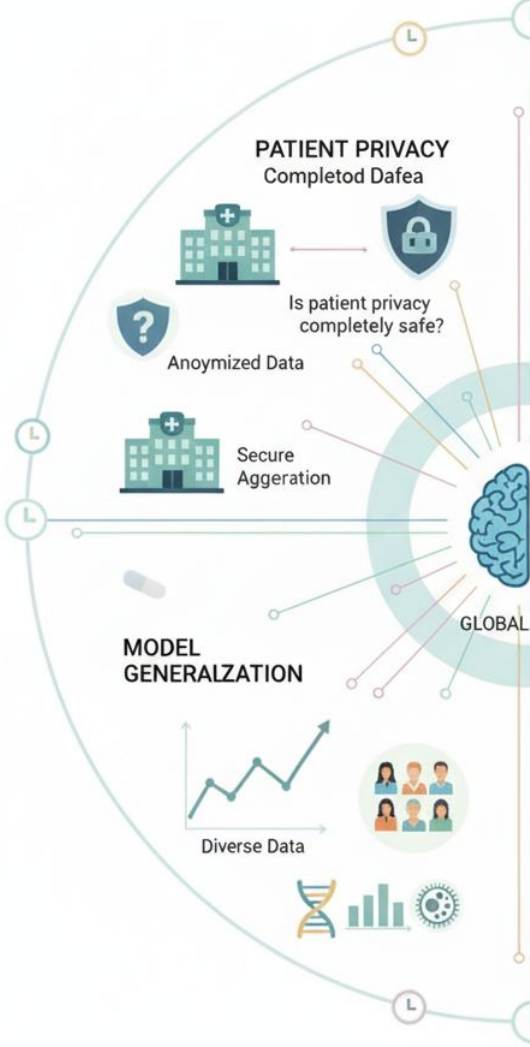


SURF

Beyond Encryption

1. Privacy not entirely safe with Secure Aggregation
   a. Data flows into the models and the model is able to **memorize** PII data

   b. Even if individual updates are protected, final aggregated model can ***leak*** information

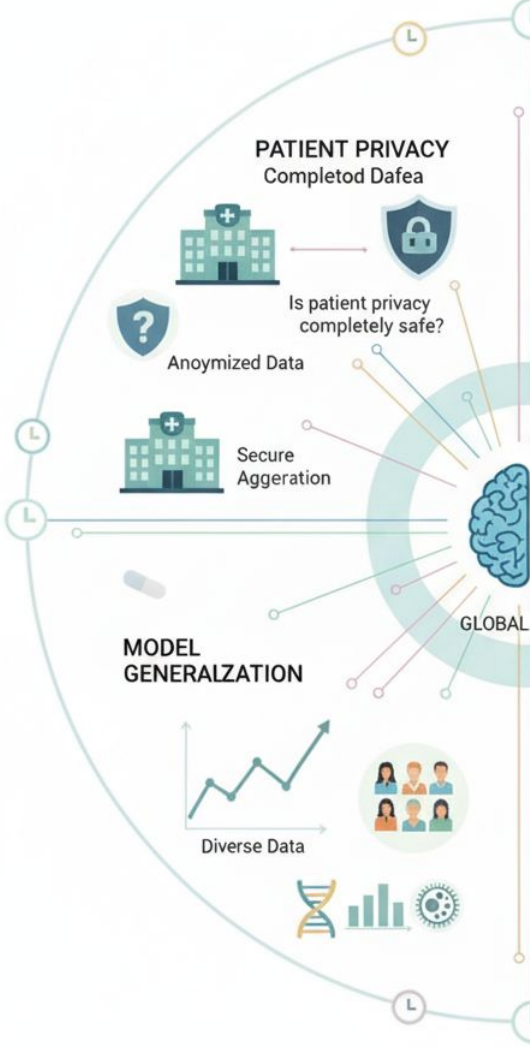


Beyond Encryption

1. Privacy not entirely safe with Secure Aggregation
   a. Data flows into the models and the model is able to **memorize** PII data

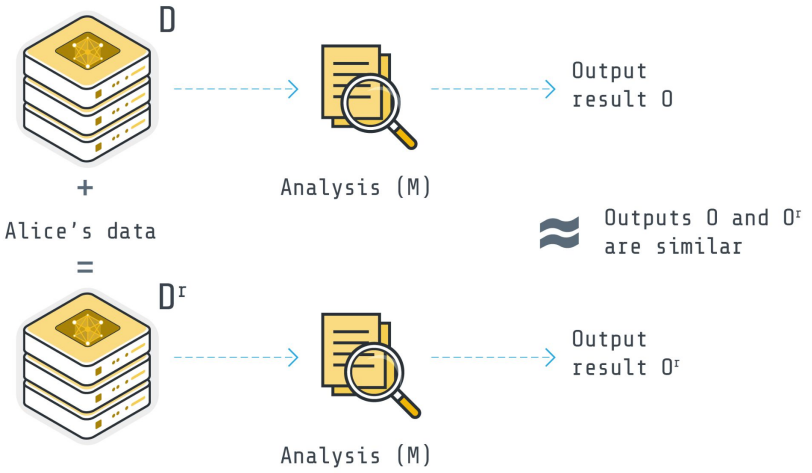
   b. Even if individual updates are protected, final aggregated model can ***leak*** information

2. **Differential Privacy**
   a. How to obscure individual updates?
      i. PII detection and redaction
      ii. Psuedoanonymization
      iii. Aggressive Dropout
      iv. Federated Averaging



Differential Privacy

Analysis is not altered by presence or absence of specific data points



Differential Privacy

Analysis is not altered by presence or absence of specific data points

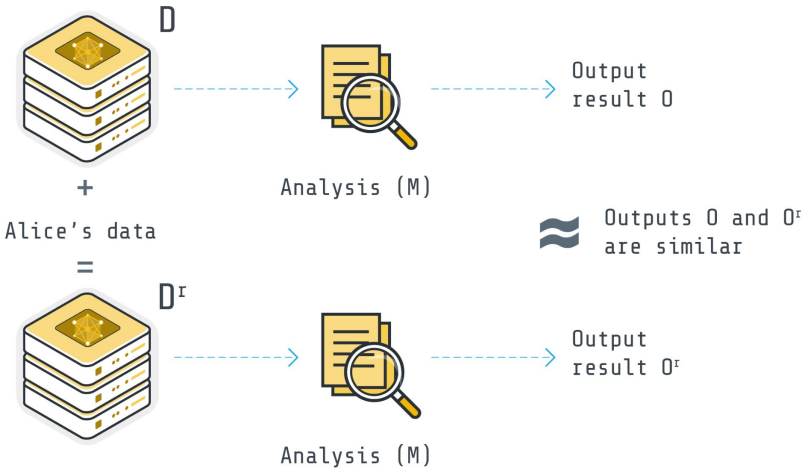
1. **Data level DP**
   Any model trained on this data will also be differentially private

2. **Model Training DP**
   Most common, if the model is differentially private, then the outputs are also differentially private.

   Label only

   Full training

3. **Predictions only DP**



Differential Privacy

Analysis is not altered by presence or absence of specific data points

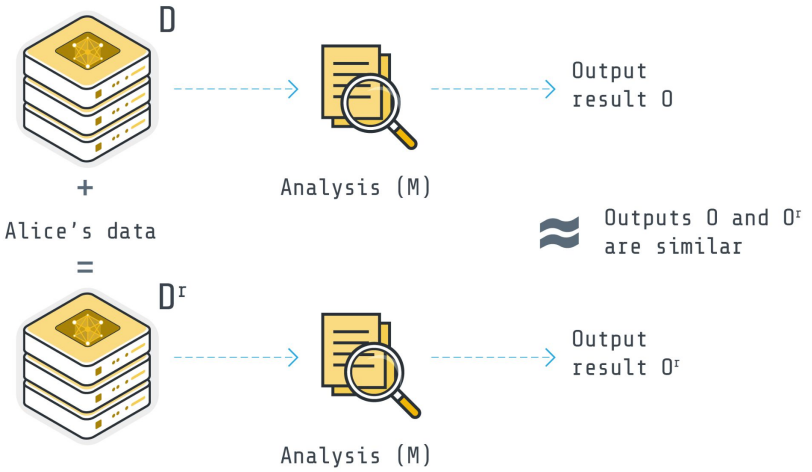
1. **Data level DP**
   Hardest; think of synthetic data

2. **Model Training DP**

   Label only; looser bound than full training, think of protecting users who clicked on an ad

   Full training; ***core subject***

3. **Predictions only DP**
   Easiest; only share model with trusted users
   Perturb outputs

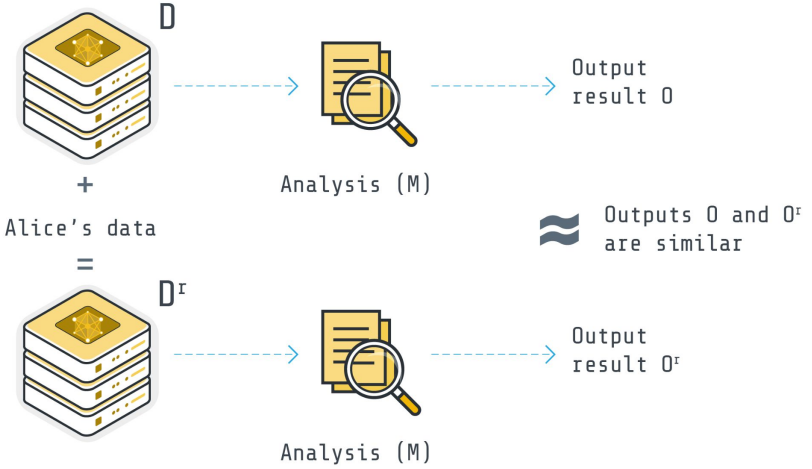


Differential Privacy

Analysis is not altered by presence or absence of specific data points

Full Model Training DP; Gradient based

1. **Inject Noise to pretrained weights**
   Or add noise at the end of training to the weights
2. **Perturb or modify the loss/objective**
3. **Gradient Noise Injection**
   DP-SGD (Abadi et al., 2016); Moments accountant and tighter privacy bounds



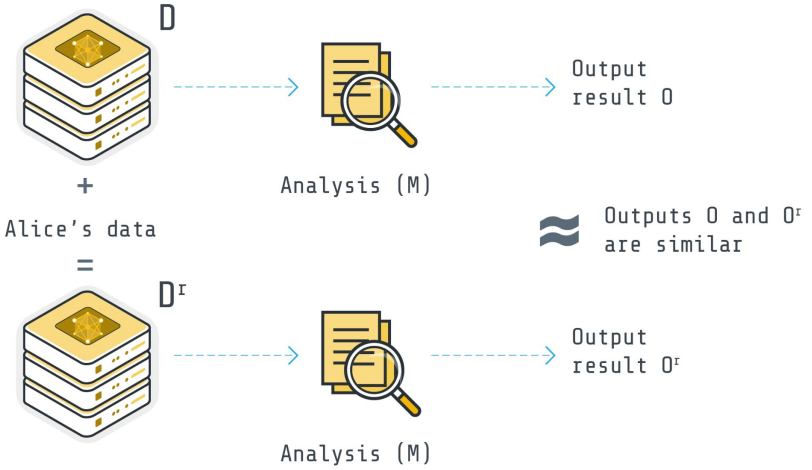
Differential Privacy

Analysis is not altered by presence or absence of specific data points

Full Model Training DP; Gradient based

1. **Inject Noise to pretrained weights**
   Or add noise at the end of training to the weights
2. **Perturb or modify the loss/objective**
3. **Gradient Noise Injection**
   DP-SGD (Abadi et al., 2016); Moments accountant and tighter privacy bounds

    1. Per example gradient L2-norm clipping
    2. Add noise (Gaussian or Laplacian) to the gradients

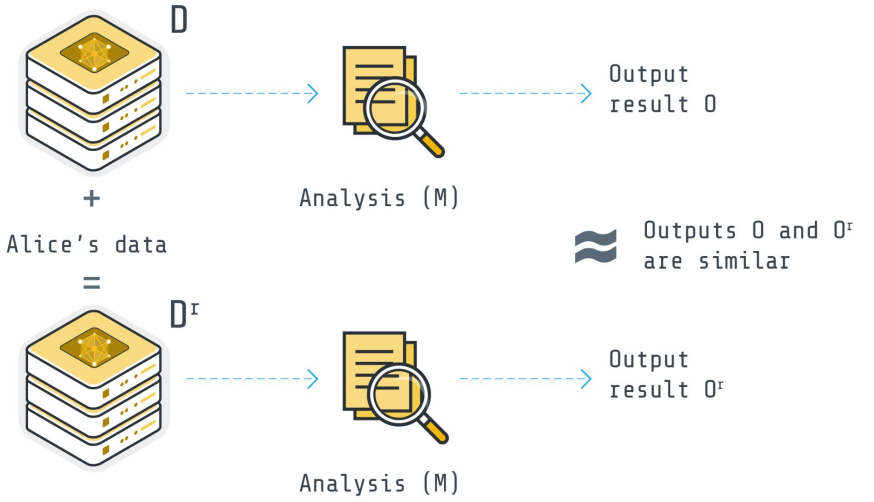


Differential Privacy

Gradient Noise Injection

1. Per example gradient L2-norm clipping

2. Add noise (Gaussian or Laplacian) to the gradients



Differential Privacy

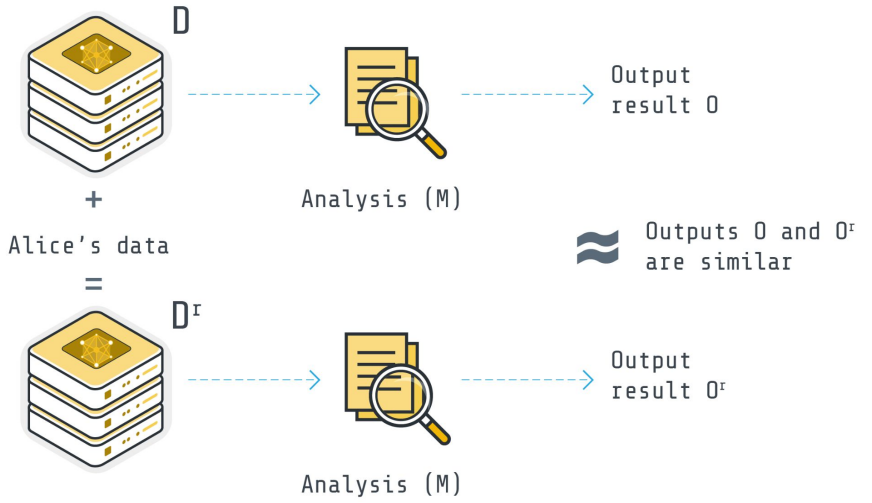
Gradient Noise Injection

1. Per example gradient L2-norm clipping

2. Add noise (Gaussian or Laplacian) to the gradients

M is considered epsilon-differentially private if for two datasets differ by one datapoint

Such that the probability of producing a specific output is (nearly) the same



Differential Privacy

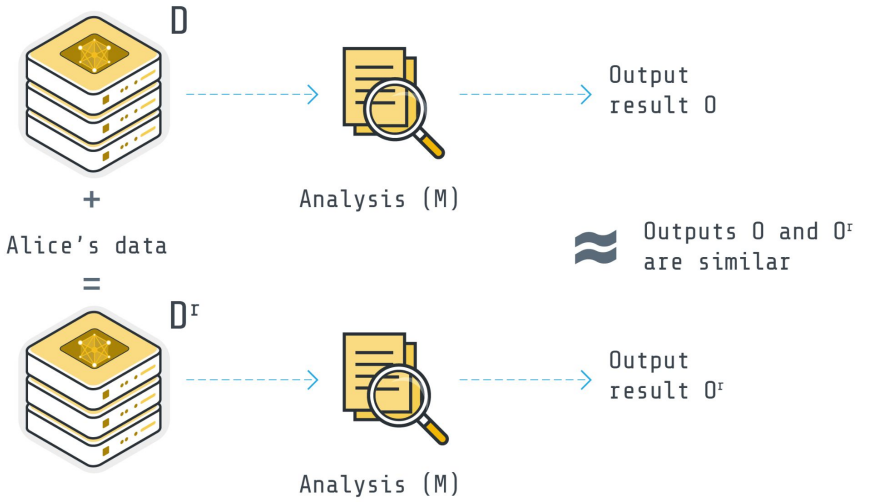
Gradient Noise Injection

1. Per example gradient L2-norm clipping
2. Add noise (Gaussian or Laplacian) to the gradients

M is considered epsilon-differentially private if for two datasets differ by one datapoint

Such that the probability of producing a specific output is (nearly) the same

$$\mathbb{P}[M(D) = x] \leq e^{\epsilon} \cdot \mathbb{P}[M(D') = x]$$



Differential Privacy

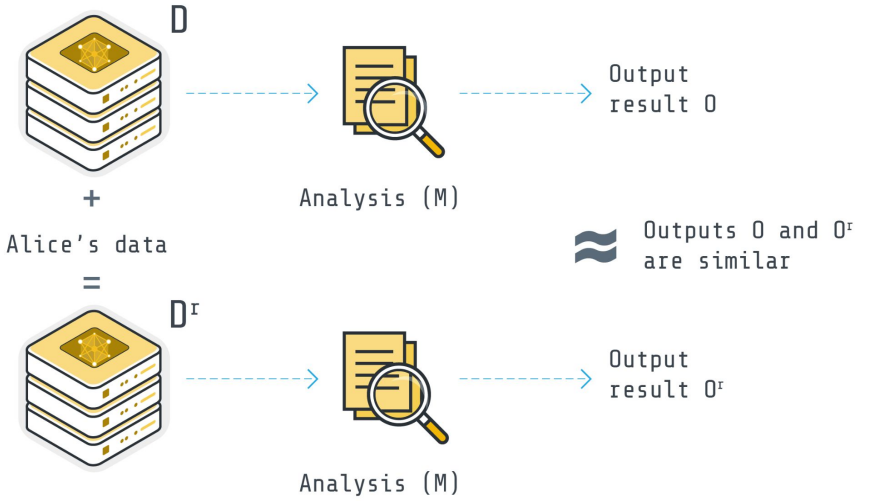
Gradient Noise Injection

1. Per example gradient L2-norm clipping
2. Add noise (Gaussian or Laplacian) to the gradients

M is considered epsilon-differentially private if for two datasets differ by one datapoint

Such that the probability of producing a specific output is (nearly) the same

$$\mathbb{P}[M(D) = x] \leq e^{\epsilon} \cdot \mathbb{P}[M(D') = x] + \delta$$



Differential Privacy

Gradient Noise Injection

1. Per example gradient L2-norm clipping

2. Add noise (Gaussian or Laplacian) to the gradients

Algorithm 1 DP-SGD algorithm

Input: Training data, consisting of features $X := \{x_1, x_2, ..., x_N\}$ and labels $Y := \{y_1, y_2, ..., y_N\}$.
$f(x;\theta)$ is the model applied to an input x and parameterized by θ.
$L(y, y')$ is the loss function for label y and prediction y'.
SGD hyperparameters: η learning rate, T number of iterations, B batch size.
DP hyperparameters: C clipping norm, σ noise level, δ (used only for privacy accounting).
Output: θ_T final model parameters
$\theta_0 \leftarrow$ randomly initialized values
for $t \leftarrow 1$ to T **do**
Randomly sample a batch B_t with sampling probability B/N for each data point.
Data are sampled with replacement for each batch.
for $i \in B_t$ **do**
$g_t(x_i) \leftarrow \nabla_{\theta_t} L(y_i, f(x_i;\theta_t))$ ▷ Compute per-example gradient wrt the weights
$g_t(x_i) \leftarrow g_t(x_i) / \max(1, \frac{||g_t(x_i)||_2}{C})$ ▷ Clip the per-example gradient
$\bar{g}_t \leftarrow \frac{1}{B}(\sum_i g_t(x_i) + \mathcal{N}(0, \sigma^2 C^2 \mathbb{1}))$ ▷ Add noise
$\theta_{t+1} \leftarrow \theta_t - \eta \bar{g}_t$ ▷ Gradient descent step

Differential Privacy

Mainly Two Guarantees

1. Forward pass on a private output **remains** private
2. Privacy budget is cumulative; we can track the total amount of *privacy* used over the model's training

Algorithm 1 DP-SGD algorithm

Input: Training data, consisting of features $X := \{x_1, x_2, ..., x_N\}$ and labels $Y := \{y_1, y_2, ..., y_N\}$.
$f(x; \theta)$ is the model applied to an input x and parameterized by θ.
$L(y, y')$ is the loss function for label y and prediction y'.
SGD hyperparameters: η learning rate, T number of iterations, B batch size.
DP hyperparameters: C clipping norm, σ noise level, δ (used only for privacy accounting).
Output: θ_T final model parameters
$\theta_0 \leftarrow$ randomly initialized values
for $t \leftarrow 1$ to T **do**
Randomly sample a batch B_t with sampling probability B/N for each data point.
Data are sampled with replacement for each batch.
for $i \in B_t$ **do**
$g_t(x_i) \leftarrow \nabla_{\theta_t} L(y_i, f(x_i; \theta_t))$ ▷ Compute per-example gradient wrt the weights
$g_t(x_i) \leftarrow g_t(x_i) / \max(1, \frac{||g_t(x_i)||_2}{C})$ ▷ Clip the per-example gradient
$\bar{g}_t \leftarrow \frac{1}{B}(\sum_i g_t(x_i) + \mathcal{N}(0, \sigma^2 C^2 \mathbb{1}))$ ▷ Add noise
$\theta_{t+1} \leftarrow \theta_t - \eta \bar{g}_t$ ▷ Gradient descent step

Differential Privacy

Gradient Noise Injection

1. Per example gradient L2-norm clipping
2. Add noise (Gaussian or Laplacian) to the gradients





Caveats

1. Not easy to process per-example gradients for clipping; auto-diff libraries don't really like this
2. Inefficient per-example gradient clipping can make model training slow
3. Loss of *utility*; loss of performance
4. More to tune; epsilon and delta

SURF

Differential Privacy $\mathbb{P}[M(D) = x] \leq e^{\epsilon} \cdot \mathbb{P}[M(D') = x] + \delta$

Mainly Two Guarantees

1. Forward pass on a private output **remains** private
2. Privacy budget is cumulative; we can track the total amount of *privacy* used over the model's training

Differential Privacy $\mathbb{P}[M(D) = x] \leq e^{\epsilon} \cdot \mathbb{P}[M(D') = x] + \delta$

Mainly Two Guarantees

1. Forward pass on a private output **remains** private
2. Privacy budget is cumulative; we can track the total amount of *privacy* used over the model's training

Accounting

ϵ represents the maximum privacy budget we are willing to spend about any specific data point

1. Every datapoint interaction with the model incurs *privacy loss*
2. In (ϵ, δ) -differential privacy we define the bounds a low epsilon (<1.0) is privacy strict, while a larger epsilon is more permissive

SURF

Differential Privacy $\mathbb{P}[M(D) = x] \leq e^{\epsilon} \cdot \mathbb{P}[M(D') = x] + \delta$

Mainly Two Guarantees

1. Forward pass on a private output **remains** private
2. Privacy budget is cumulative; we can track the total amount of *privacy* used over the model's training

Accounting

ϵ represents the maximum privacy budget we are willing to spend about any specific data point

1. Every datapoint interaction with the model incurs *privacy loss*
2. In (ϵ, δ) -differential privacy we define the bounds a low epsilon (<1.0) is privacy strict, while a larger epsilon is more permissive

$$L = \log\left(\frac{\mathbb{P}[\text{output}|D]}{\mathbb{P}[\text{output}|D']}\right)$$

Differential Privacy $\mathbb{P}[M(D) = x] \leq e^{\epsilon} \cdot \mathbb{P}[M(D') = x] + \delta$

Mainly Two Guarantees

1. Forward pass on a private output **remains** private
2. Privacy budget is cumulative; we can track the total amount of *privacy* used over the model's training

Accounting

$$L = \log\left(\frac{\mathbb{P}[\text{output}|D]}{\mathbb{P}[\text{output}|D']}\right)$$

ϵ represents the maximum privacy budget we are willing to spend about any specific data point

1. Every datapoint interaction with the model incurs *privacy loss*
2. In (ϵ, δ) -differential privacy we define the bounds a low epsilon (<1.0) is privacy strict, while a larger epsilon is more permissive
3. **Moments accountant** keeps a running total by considering the privacy loss at every batch training step as a random variable
   a. Noise injection
   b. Mini-batch sampling

SURF

Differential Privacy $\mathbb{P}[M(D) = x] \leq e^{\epsilon} \cdot \mathbb{P}[M(D') = x] + \delta$

Mainly Two Guarantees

1. Forward pass on a private output **remains** private
2. Privacy budget is cumulative; we can track the total amount of *privacy* used over the model's training

Accounting

ϵ represents the maximum privacy budget we are willing to spend about any specific data point

1. Every datapoint interaction with the model incurs *privacy loss*
2. In (ϵ, δ) -differential privacy we define the bounds a low epsilon (<1.0) is privacy strict, while a larger epsilon is more permissive
3. **Moments accountant** keeps a running total by considering the privacy loss at every batch training step as a random variable
   a. Noise injection
   b. Mini-batch sampling

$$L = \log\left(\frac{\mathbb{P}[\text{output}|D]}{\mathbb{P}[\text{output}|D']}\right)$$

$$L \leq \epsilon$$

ϵ is the maximum possible value that the privacy loss random variable can take

SURF

Differential Privacy $\mathbb{P}[M(D) = x] \leq e^{\epsilon} \cdot \mathbb{P}[M(D') = x] + \delta$

Mainly Two Guarantees

1. Forward pass on a private output **remains** private
2. Privacy budget is cumulative; we can track the total amount of *privacy* used over the model's training

Accounting

ϵ represents the maximum privacy budget we are willing to spend about any specific data point

1. Every datapoint interaction with the model incurs *privacy loss*
2. In (ϵ, δ) -differential privacy we define the bounds a low epsilon (<1.0) is privacy strict, while a larger epsilon is more permissive
3. **Moments accountant** keeps a running total by considering the privacy loss at every batch training step as a random variable
   a. Noise injection
   b. Mini-batch sampling

$$L = \log\left(\frac{\mathbb{P}[\text{output}|D]}{\mathbb{P}[\text{output}|D']}\right)$$

So don't just add the epsilon values that's the **worst case privacy loss**, we want a tighter bound!

Use a moment generating function (MGF)

SURF

Differential Privacy $\mathbb{P}[M(D) = x] \leq e^{\epsilon} \cdot \mathbb{P}[M(D') = x] + \delta$

Mainly Two Guarantees

1. Forward pass on a private output **remains** private
2. Privacy budget is cumulative; we can track the total amount of *privacy* used over the model's training

Accounting

ϵ represents the maximum privacy budget we are willing to spend about any specific data point

1. Every datapoint interaction with the model incurs *privacy loss*
2. In (ϵ, δ) -differential privacy we define the bounds a low epsilon (<1.0) is privacy strict, while a larger epsilon is more permissive
3. **Moments accountant** keeps a running total by considering the privacy loss at every batch training step as a random variable
   a. Noise injection
   b. Mini-batch sampling

$$L = \log\left(\frac{\mathbb{P}[\text{output}|D]}{\mathbb{P}[\text{output}|D']}\right)$$

How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy

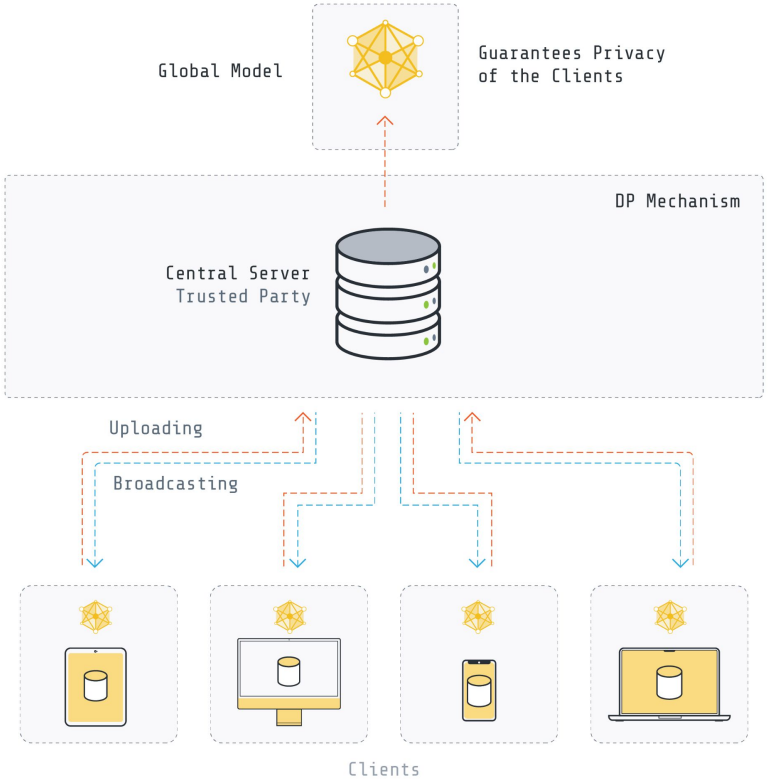
https://arxiv.org/abs/2303.00654

SURF

Differential Privacy in FL

Two Types of DP in FL

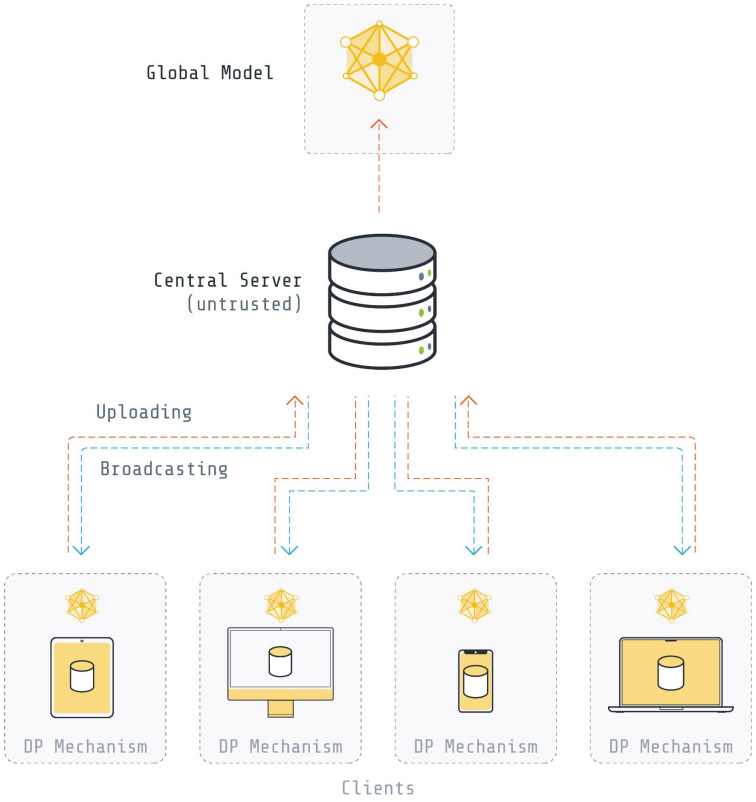
1. Central DP



Differential Privacy in FL

Two Types of DP in FL

1. Central DP
2. Local DP



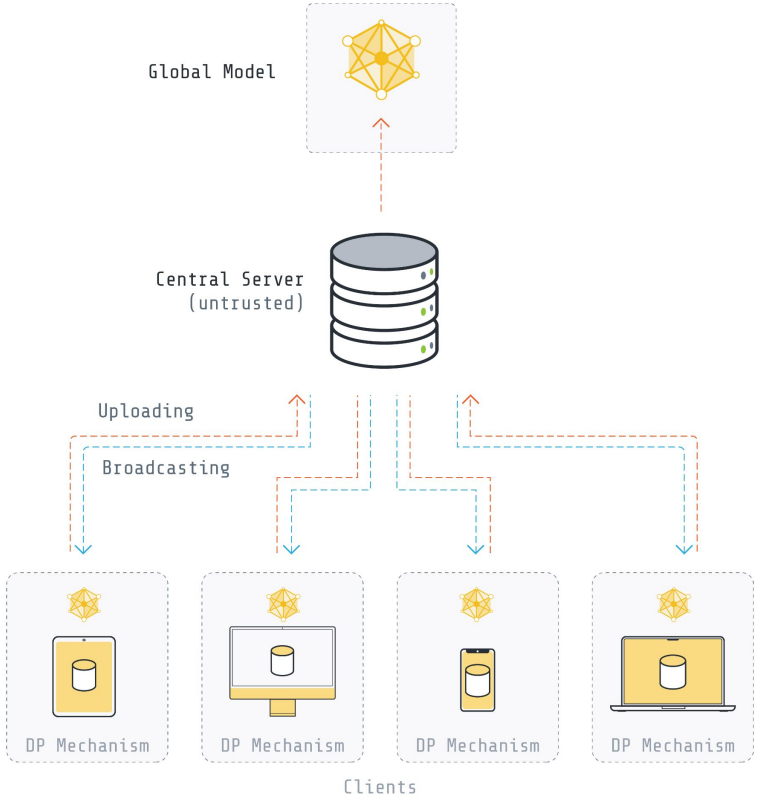
Differential Privacy in FL

Two Types of DP in FL

1. Central DP
2. Local DP

User-Level DP; Protect the *client*

for global round $t \leftarrow 1$ to T **do**
  Randomly sample a subset of users $\mathcal{S}^t$ of B_c users
  for each user $u \in \mathcal{S}^t$ **do**
    Initialize $\omega_u^0 = \theta_{t-1}$
    for local iteration $k \leftarrow 1$ to K **do**
      Sample minibatch $\mathcal{B}_u^{(t,k)} \subset \mathcal{D}_u$ of B_m examples.
      $g_u^{(t,k)} \leftarrow \frac{1}{B_m} \sum_{j \in \mathcal{B}_u^{(t,k)}} \nabla_{\omega_u} L(y_j, f(x_j; \omega_u^{k-1}))$
      $\omega_u^k \leftarrow \omega_u^{k-1} - \eta_c g_u^{(t,k)}$
    $\Delta_u^t \leftarrow \omega_u^0 - \omega_u^K$
    $\tilde{\Delta}_u^t \leftarrow \Delta_u^t / \max(1, \frac{||\Delta_u^t||_2}{C})$
  $\bar{\Delta}^t \leftarrow \frac{1}{B_c}(\sum_{i \in \mathcal{S}^t} \tilde{\Delta}_i^t + \mathcal{N}(0, \sigma^2 C^2 \mathbb{1}))$
  $\theta_t \leftarrow \theta_{t-1} - \eta_s \bar{\Delta}^t$



Privacy-Utility Trade Off



SURF

High Performance Machine Learning Group

VaultGemma

https://research.google/blog/vaultgemma-the-worlds-most-capable-differentially-private-llm/

1. For a given compute budget, privacy budget, and data budget, what is the optimal training configuration to achieve the lowest possible training loss?

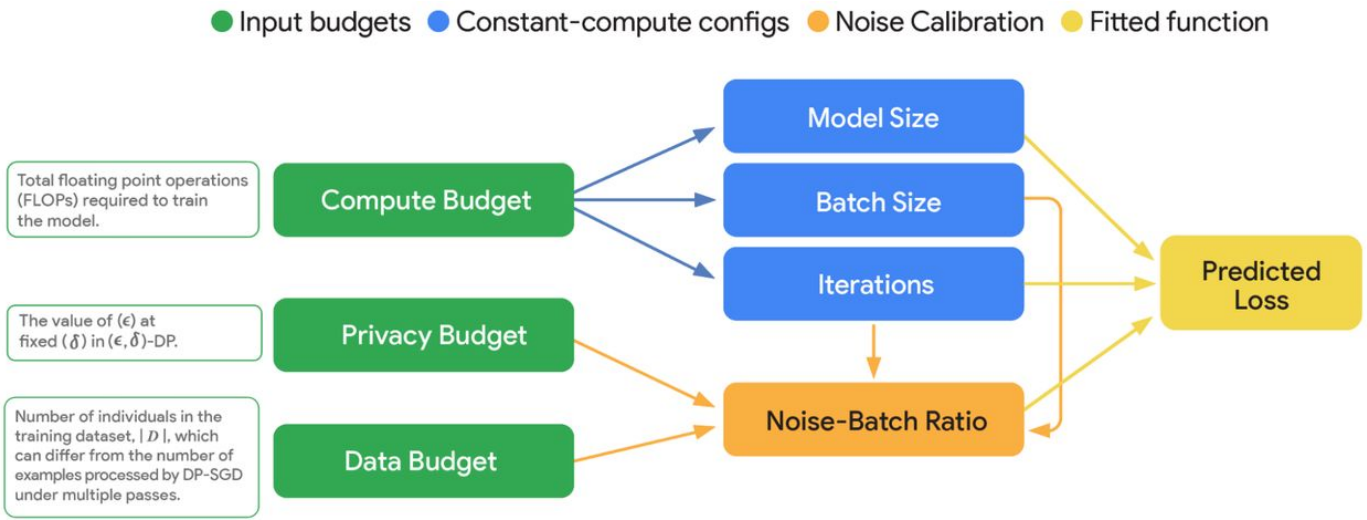
VaultGemma was trained with a *sequence*-level DP guarantee of ($\varepsilon \leq 2.0$, $\delta \leq$ 1.1e-10), where a sequence consists of 1024 consecutive tokens

https://research.google/blog/vaultgemma-the-worlds-most-capable-differentially-private-llm/

VaultGemma

1. For a given compute budget, privacy budget, and data budget, what is the optimal training configuration to achieve the lowest possible training loss?

VaultGemma was trained with a *sequence*-level DP guarantee of ($\varepsilon \leq 2.0$, $\delta \leq$ 1.1e-10), where a sequence consists of 1024 consecutive tokens

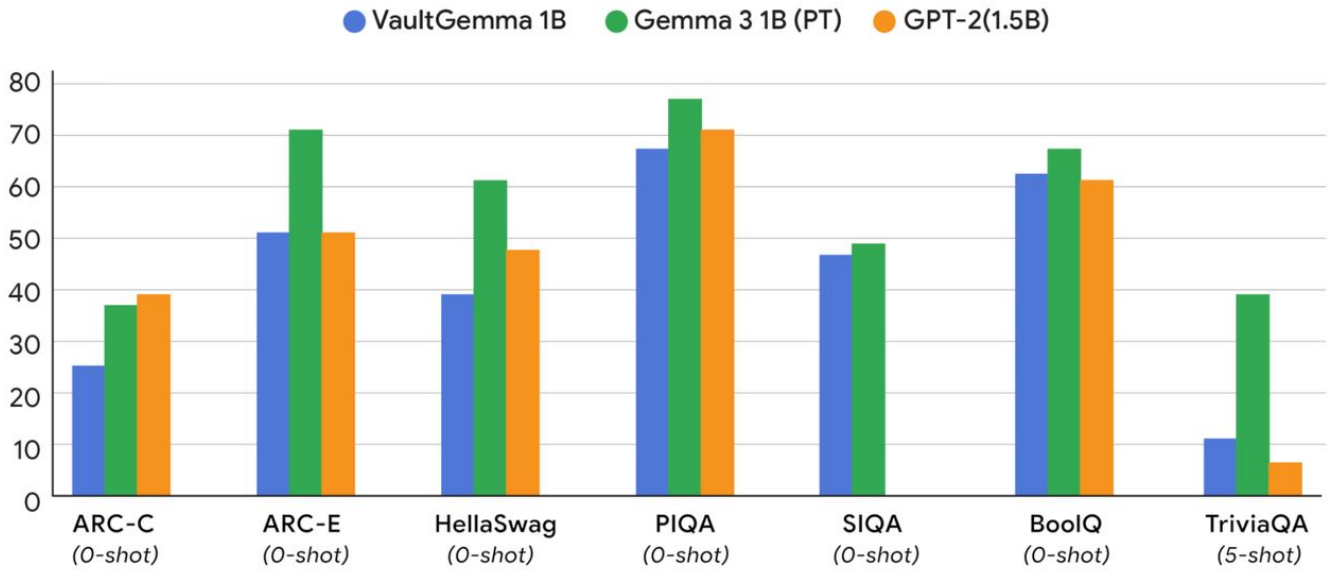


https://research.google/blog/vaultgemma-the-worlds-most-capable-differentially-private-llm/

VaultGemma

1. For a given compute budget, privacy budget, and data budget, what is the optimal training configuration to achieve the lowest possible training loss?

VaultGemma was trained with a *sequence*-level DP guarantee of ($\varepsilon \leq 2.0$, $\delta \leq$ 1.1e-10), where a sequence consists of 1024 consecutive tokens



VaultGemma

https://research.google/blog/vaultgemma-the-worlds-most-capable-differentially-private-llm/

1. For a given compute budget, privacy budget, and data budget, what is the optimal training configuration to achieve the lowest possible training loss?

VaultGemma was trained with a *sequence*-level DP guarantee of ($\varepsilon \leq 2.0$, $\delta \leq$ 1.1e-10), where a sequence consists of 1024 consecutive tokens

Data filter in VaultGemma

1. Unwanted or unsafe utterances
2. Personal information or other sensitive data
3. Decontaminate evaluation sets from the pretraining data mixture
4. Reduce the risk of recitation by minimizing the proliferation of sensitive outputs

Locally trained DP-SLM

1. Very Difficult to tune the model
2. Maybe try only fine tuning on private data
3. finding a set of hyperparameters that yields a model with both a meaningful privacy guarantee (a low ε) and acceptable task performance is a non-trivial task.
4. Utility gap will always exist



Questions?



SURF